

События сервисного модуля и их обработчики

В жизненном цикле сервисного модуля существует несколько событий, на которые прикладной разработчик может создавать собственные обработчики. В этой заметке приведён список таких событий, момент их возникновения, примеры использования, а также требуемые условия для работы обработчика.

События модуля

События **OnEndLoadKernelModules** и **OnKernelCleanUp** запрещено использовать в прикладных проектах!

- **OnEndLoadKernelModules (C++)** – происходит после получения списка инициализаторов. Вызывается для каждого модуля в прямом порядке зависимостей — от базовых модулей к высокоуровневым.
- **OnKernelCleanup (C++)** – отвечает за освобождение ресурсов, задействованных после загрузки модуля. Вызывается для каждого модуля, в обратном порядке зависимостей — от высокоуровневых к базовым модулям.
- **OnEndLoadModules/OnEndAllLoadModules (C++ / Python)** происходит после загрузки всех модулей конкретного сервиса приложения в прямом порядке зависимостей — от базовых модулей к высокоуровневым.
- **OnCleanup (C++/Python)** – отвечает за освобождение ресурсов, задействованных после загрузки модуля. Функция вызывается для каждого модуля, в обратном порядке — от высокоуровневых к базовым модулям.

У динамических инициализаторов есть еще два события:

- **OnCheck/OnCheckModules(C++ / Python)** – может вызываться для проверки состояния сервиса. Например, платформенный модуль Db Manager через этот коллбэк выводит текущее состояние баз данных. Если нет необходимости в проверке, то данный метод реализовывать не требуется.
- **OnUnload(C++ / Python)** – событие начала выгрузки модулей.

Наличие инициализаторов не обязательно. Переопределяйте только те события, в которых вам реально нужно что-то делать.



Примечание

При типовой настройке сервиса событие OnUnload не вызывается. Как правило, работа процессов сервиса заканчивается вызовом метода QuickExit().

Все обработчики возвращают логическое значение, обозначающее успешность/неуспешность обработки события и могут сообщать дополнительную информацию через параметр answer.

Пример ответа через answer:

```
1 // инициализирует модуль
2 void InitMyModule();
3 // чистит временные данные модуля
4 void CleanupMyModule();
5 // проверяет состояние функционала модуля
6 void CheckMyModule();
7 struct MyModuleInitializer : Id::ModuleInitializerHandle
8 {
9     StringView Name() const noexcept override
10    {
11        return L"MyModule"_sv;
12    }
13    bool OnEndLoadModules( EndLoadEventAnswer& answer ) const
14    override
15    {
16        try
```

```

17         InitMyModule();
18         return true;
19     }
20     catch( Exception const& e )
21     {
22         answer.mErrorMessage = e.ErrorMessage();
23     }
24     return false;
25 }
26 bool OnCleanup( CleanupEventAnswer& ) const override
27 {
28     try
29     {
30         CleanupMyModule();
31         return true;
32     }
33     catch( Exception const& e )
34     {
35         answer.mErrorMessage = e.ErrorMessage();
36     }
37     return false;
38 }
39 bool OnCheck( CheckEventAnswer& answer ) const override
40 {
41     try
42     {
43         answer.mModuleName = L"MyModule";
44         answer.mErrorMessage.clear();
45         CheckMyModule();
46         answer.mIsModuleOK = true;
47     }
48     catch( const Exception& e )
49     {
50         answer.mErrorMessage = e.ErrorMessage();
51         answer.mIsModuleOK = false;
52         return true;
53     }
54     return true;
55 }
56 };
57

```

Также можно не использовать answer, а прокидывать исключения дальше:

```

1 struct MyModuleInitializer : ld::ModuleInitializerHandle
2 {
3     StringView Name() const noexcept override
4     {
5         return L"MyModule"_sv;
6     }
7     bool OnEndLoadModules( EndLoadEventAnswer& answer ) const override
8     {
9         InitMyModule();
10        return true;
11    }

```

```

12     bool OnCleanup( CleanupEventAnswer& ) const override
13     {
14         CleanupMyModule();
15         return true;
16     }
17     bool OnCheck( CheckEventAnswer& answer ) const override
18     {
19         try
20         {
21             answer.mModuleName = L"MyModule";
22             answer.mErrorMessage.clear();
23             CheckMyModule();
24             answer.mIsModuleOK = true;
25         }
26         catch( const Exception& e )
27         {
28             answer.mErrorMessage = e.ErrorMessage();
29             answer.mIsModuleOK = false;
30             return true;
31         }
32         return true;
33     }
34 };

```

Создание обработчика на Python

1. В сервисном модуле, где будет создан обработчик на событие, требуется установить зависимость от сервисного модуля Python Core (поставляется с [SBIS SDK](#)).
2. В корне директории сервисного модуля создайте файл on_event.py.
3. Добавьте обработчики событий. Они должны называться так же, как и события модуля. Из обработчика может быть возвращено True, иначе False – в таком случае процесс загрузки будет прерван с ошибкой.

Пример 1. Используем OnEndAllLoadModules для регистрации точки входа.

```

1 def OnEndAllLoadModules(answer):
2     try:
3         RegisterEntryPoint( TestPyEntryPoint( "TestCasePy" ) )
4         RegisterEntryPoint( TestPyEntryPoint2( "TestCasePy2" ) )
5     except Exception:
6         pass
7     return True

```

Пример 2. Подписка на событие для обновления регламентов.

```

1 import sbis
2 def event_subscription():
3
4     # Подписка на событие в облаке.
5     import event
6     subscription = event.Subscription()
7     subscription.channel = 'versionmanager.applypackage'
8     subscription.durable = True
9     subscription.for_instance = True
10    subscription.user_login = 'online_service'
11    subscription.SetCallback("RegulationService.PackageCallback",
    event._MsgBody_)

```

```

12     subscription.Add()
13
14     def OnEndAllLoadModules(answer):
15         event_subscription()
16         return True

```

Создание обработчика на C++

Для создания обработчика на C++ необходимо реализовать наследника класса **ModuleInitializerHandle**, переопределить метод **Name** и обработчики необходимых событий. Возвращаемое значение метода **Name** должно В ТОЧНОСТИ совпадать с именем модуля, указанным в **s3mod**-файле. После этого необходимо зарегистрировать инициализатор в специальном глобальном реестре инициализаторов с помощью макроса **REGISTER_MODULE_INITIALIZER**.

Интерфейс

```

1 namespace ld
2 {
3     class LD_DLLCLASS ModuleInitializerHandle : public
4         AbstractModuleHandle
5     public:
6         bool OnEndLoadKernelModules( EndLoadEventAnswer& ) const override;
7         bool OnEndLoadModules( EndLoadEventAnswer& ) const override;
8         bool OnKernelCleanup( CleanupEventAnswer& ) const override;
9         bool OnCleanup( CleanupEventAnswer& ) const override;
10        bool OnCheck( CheckEventAnswer& ) const override;
11        bool OnUnload( CleanupEventAnswer& ) const override;
12        bool HasNativeInitializer() const noexcept override
13        {
14            return True;
15        }
16 };

```

Пример обработчика события

```

1 class Db SQLiteInitializer : public ld::ModuleInitializerHandle
2
3 public:
4     StringView Name() const noexcept override
5     {
6         return L"Db SQLite"_sv;
7     }
8
9     bool OnEndLoadModules( EndLoadEventAnswer& answer ) const override
10    {
11        // Обработчик события
12        ...
13    }
14 };
15 REGISTER_MODULE_INITIALIZER( Db SQLiteInitializer )
16
17 // namespace sqlite

```